Breast Cancer Prediction Prototype

Project Guide

Breast Cancer Prediction Prototype

A Python and Streamlit learning prototype for entering the 30 measurements from the Wisconsin Diagnostic Breast Cancer (WDBC) dataset and viewing a model prediction. The training script compares exactly three classifiers: Logistic Regression, SVM, and Random Forest. It selects one using cross-validation only; the app loads that saved model and never trains during a prediction.

> **Clinical limitation:** This is an educational demonstration, not a diagnostic tool or medical device. The dataset and evaluation do not establish clinical performance. This output is not a diagnosis and must be interpreted by a qualified healthcare professional alongside clinical findings.

Clone and run

Clone the GitHub repository and enter its directory:

```
git clone https://github.com/OtieteAyebanua/brestCancerdetection.git
cd brestCancerdetection
```

Create a Python environment, install the dependencies, and start the app:

macOS/Linux:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
streamlit run app.py --server.port 3001
```

Windows PowerShell:

```
py -3.13 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

```
python -m pip install -r requirements.txt
streamlit run app.py --server.port 3001
```

Open http://localhost:3001. Stop the app with **Ctrl+C**. The Streamlit config sets the light theme but leaves the listening address and port to the host, so hosted environments can choose their own port.

The project supports Python 3.9 through 3.13. `.python-version` selects Python 3.13 on platforms that use that file.

Using the interface

1. Select **Mean**, **Standard error**, or **Worst**.
2. Enter the 10 non-negative measurements for each section. Values are in dataset units; do not convert them.
3. Complete all 30 fields and click **Get prediction**.
4. Use **Fill with demo values** to alternate between randomized benign and malignant examples selected from the training split. They are not test cases.
5. Click **Clear inputs** to reset the current session.

The app displays the predicted dataset class and both class scores. The malignant score is a model-estimated probability and is not calibrated as an individual risk estimate. Values outside the training ranges produce a warning, but the app still returns the model output.

Inputs live in the active Streamlit session only. The project has no accounts, database, assessment history, or application-level input saving. When hosted, inputs are sent to the server running Streamlit to perform inference; hosting and server logging are outside this repository's control.

Model training and selection

The training script uses scikit-learn's bundled WDBC data (569 rows, 30 features). No separate training-data download is required. To rebuild all artifacts:

```
python train_model.py
```

The procedure is fixed:

1. Make one stratified 80/20 train/test split with seed 42.
2. Run five-fold stratified cross-validation with shuffle and seed 42 on the training split only. Scaling is fitted inside each fold for Logistic Regression and SVM; Random Forest uses raw values.
3. Grid-search ROC-AUC, malignant recall, malignant precision, F1, and accuracy, refitting each model on ROC-AUC.
4. Select among models whose mean CV ROC-AUC is within 0.01 of the best. Choose the highest malignant recall, breaking ties by F1 and then simplicity: Logistic Regression, SVM, Random Forest.

5. Evaluate all three tuned models on the held-out test split once, after selection. Save the selected tuned estimator exactly as evaluated, fitted on the training split only.

No class weighting is used. The requested parameter grids are run in full.

Best cross-validation results

Metrics are means across the five training folds. Recall, precision, F1, and ROC-AUC treat malignant as the positive class.

Model	Best parameters	ROC-AUC	Recall	Precision	F1	Accuracy
Logistic Regression	`C=1`	0.9959	0.9647	0.9770	0.9703	0.9780
SVM	`kernel=rbf, C=10, gamma=0.01`	0.9964	0.9529	0.9820	0.9668	0.9758
Random Forest	`n_estimators=300, max_depth=5, min_samples_split=2`	0.9809	0.9412	0.9490	0.9442	0.9582

Selected model: Logistic Regression. All three models fall within 0.01 of the highest mean CV ROC-AUC. Logistic Regression has the highest mean malignant recall among them.

One-time held-out test results

The test split contains 114 cases (42 malignant, 72 benign). It was not used for tuning or model selection.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Confusion matrix[1]
Logistic Regression	0.9825	0.9762	0.9762	0.9762	0.9954	`[[41, 1], [1, 71]]`
SVM	0.9825	0.9762	0.9762	0.9762	0.9977	`[[41, 1], [1, 71]]`

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Confusion matrix[1]
Random Forest	0.9561	0.9512	0.9286	0.9398	0.9927	`[[39, 3], [2, 70]]`

[1] Rows are actual and columns predicted, ordered malignant then benign.

These are results for a single split of a small public benchmark dataset. They are not external validation, clinical validation, or evidence of performance on new hospitals, populations, or workflows.

Generated artifacts

Running `python train_model.py` creates or replaces:

Path	Contents
`models/breast_cancer_model.joblib`	Selected tuned pipeline, training-split demo profiles and ranges, feature names and input keys, positive-class label, and run metadata.
`models/model_evidence.json`	Every grid combination's CV scores, best parameters, selection decision, test metrics for all three models, confusion matrices, and library versions.
`models/roc_curves.png`	Held-out ROC curves for all three models.
`models/confusion_matrices.png`	Held-out confusion matrices for all three models.
`demo_cases.csv`	Six held-out rows: three benign and three malignant, all 30 measurements, true label, and selected-model prediction and score for comparison.
`models/pip-freeze.txt`	Exact Python package versions installed in the local training environment.

The application only loads `models/breast_cancer_model.joblib`. It validates the bundle schema and never retrains. Demo profiles are chosen from the training split, not the held-out test set. If a measurement is outside its training-split

minimum or maximum, the app warns that its prediction may be unreliable and still allows the prediction.

Project files

Path	Purpose
`app.py`	Streamlit input form, session-only input state, schema/range checks, and inference.
`train_model.py`	Fixed three-model grid search, cross-validation selection, holdout evaluation, plots, evidence, and demo-case export.
`requirements.txt`	Python app and training dependencies.
`.python-version`	Preferred Python version for compatible hosting platforms.
`.streamlit/config.toml`	Light theme configuration; no fixed server port.
`demo_cases.csv`	Six labeled cases from the held-out test split.
`models/`	Selected model, evidence JSON, and evaluation plots.
`models/pip-freeze.txt`	Local environment's `pip freeze` output.

Limitations

- This is a classroom prototype, not a clinical tool.
- The bundled dataset is small and cannot establish clinical validity or population-level reliability.
- Model probabilities are not calibrated patient-risk estimates.
- The interface does not provide diagnosis, treatment advice, patient accounts, storage, or history.
- Demo values are randomized perturbations of one dataset example, not the held-out demo cases and not clinical cases.